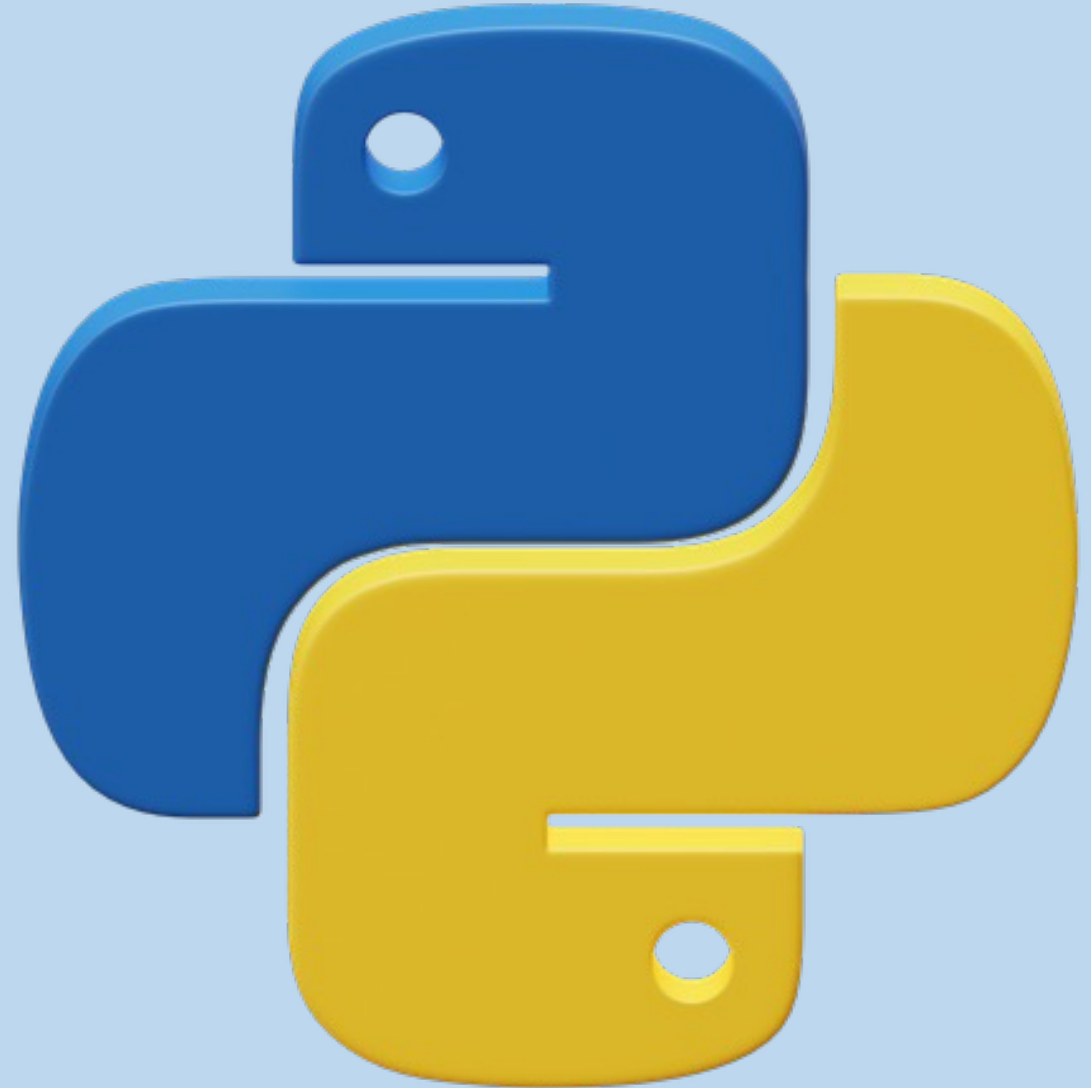


Advanced Programming

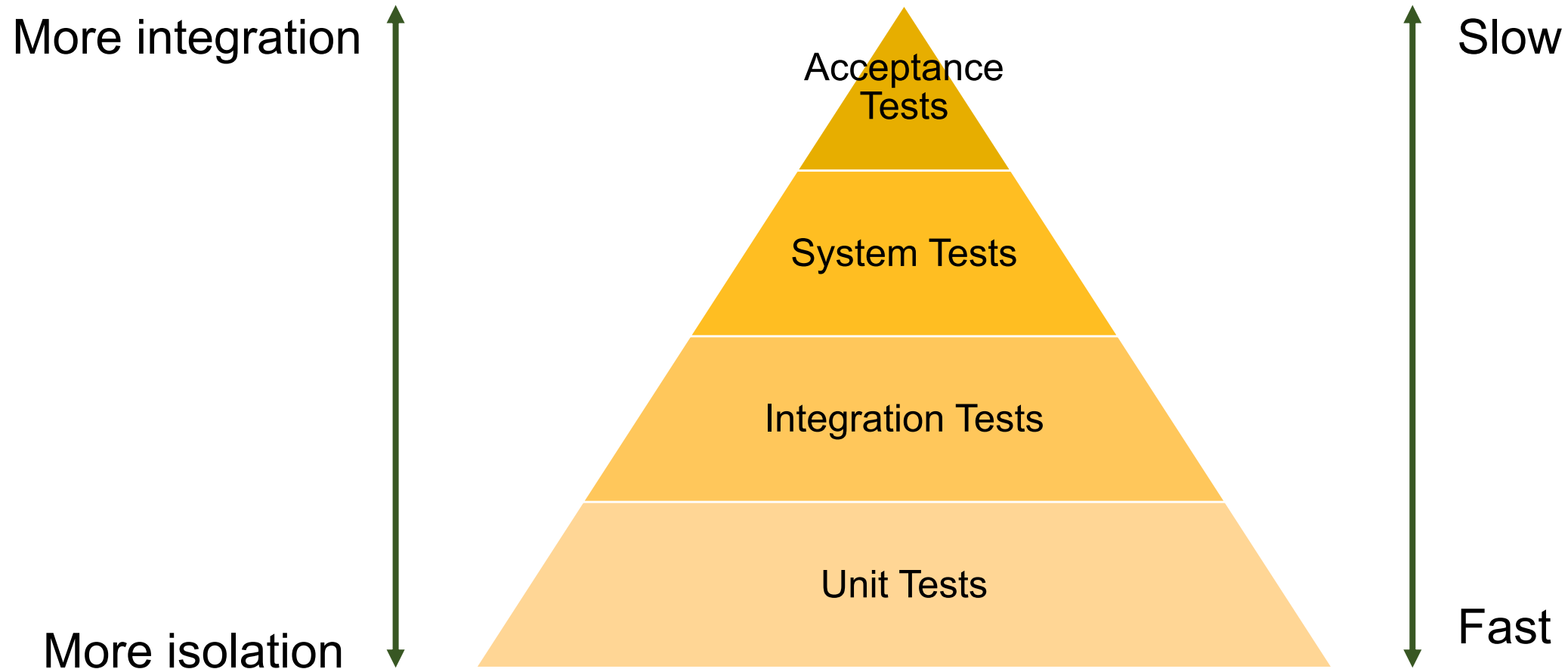
Automated Testing of Code



Today's Topics

1. Why automated testing
2. Taxonomy of Tests
3. Test Cases and Unit Tests
4. Test-Driven-Development
5. Testing Frameworks
6. Happy Paths and Edge Cases
7. Test Coverage Illusions

Testing Taxonomy



Testing Taxonomy

Unit Tests

Test a single function/method in isolation.
Fast,
deterministic, no I/O.
Mock all external dependencies
Python: pytest / unittest

Integration Tests

Two or more modules working together. E.g., DB, file system, HTTP calls involved.
Slower but more realistic.
Python example: pytest + test DB

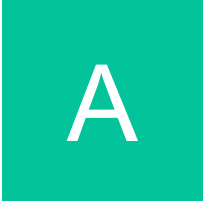
System Tests

Test the complete system end-to-end.
Simulate real user scenarios.
Check all layers together.
Tools: Selenium, Playwright

Acceptance Tests

Validate against business requirements.
Written with stakeholders.
Define DONE for a feature.
Tools: pytest-bdd

Unit Tests and The AAA-Pattern

A

Arrange

Set up test data, mocks, and preconditions.
Create the objects under test.

A

Act

Call the function / method being tested.
One action per test case.

A

Assert

Verify the outcome matches expectations.
`assert result == expected`

Unit Tests and The AAA-Pattern

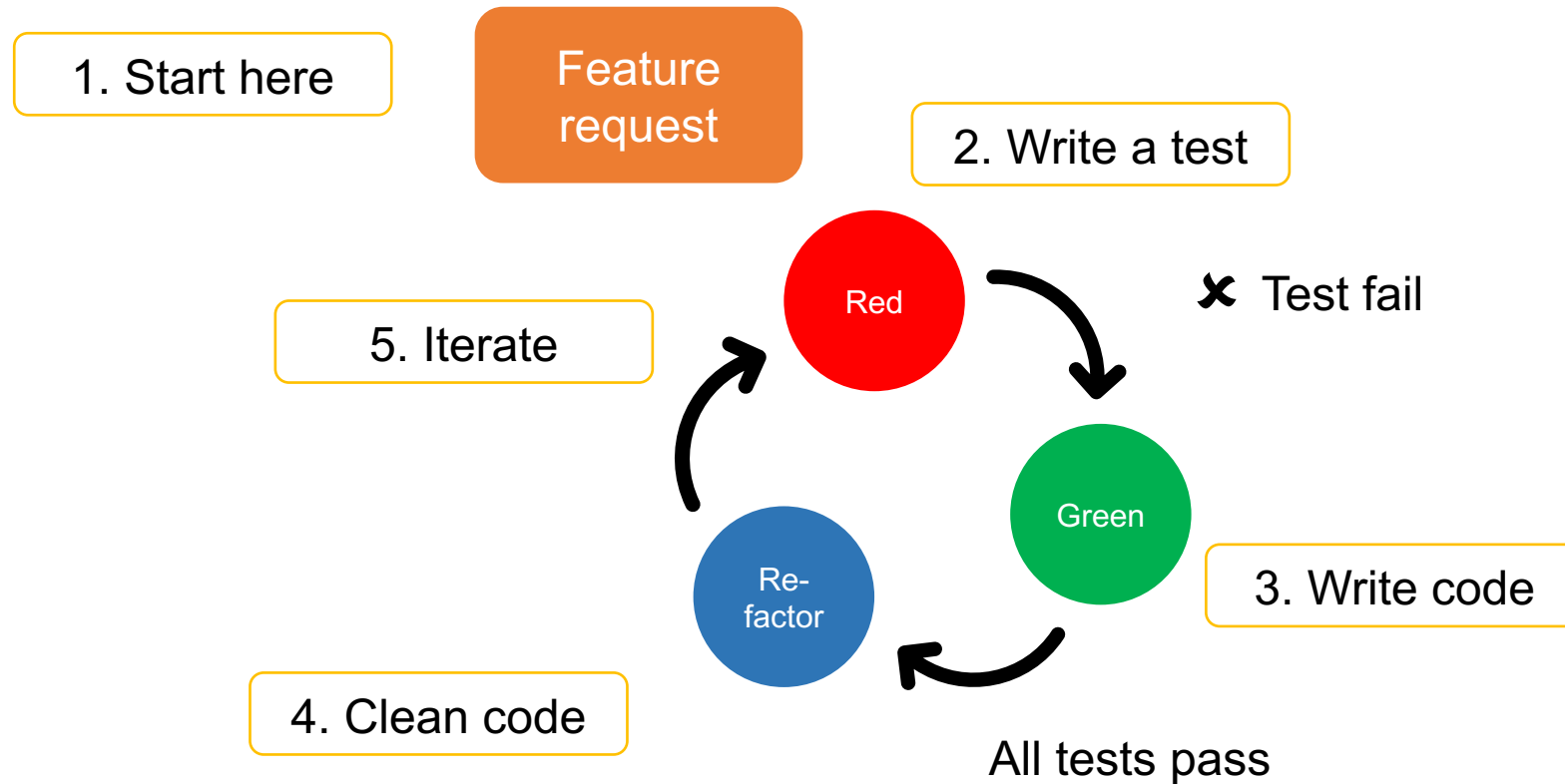
A good test is:

- Fast (ms not seconds)
- Independent (no shared state)
- Repeatable (same result every time)
- Self-validating (pass or fail)
- Timely (written before/with code)

What makes a good test?

- Testing every possible input is unrealistic. So pick a sample of expected inputs:
 - Boundary cases
 - Input that covers every branch in the tested function/class
 - Inputs from regular expected ranges and outside
- Verify a single aspect of the code with each test
- Testing code is often much longer than the code being tested

Test-Driven-Development (TDD)



[5 steps of test-driven development](#)

Test-Driven-Development

RED

Write a test
for the new
functionality.
Run it — it must FAIL.
(Nothing written yet!)

GREEN

Write the minimum
code to make the
test pass.
No extras!

REFACTOR

Improve the code
without changing
behaviour.
Tests must stay
green.

Test-Driven Development

Advantages

- Early planning leads to better program design and higher code quality
- Difficulty to test features may point to design issues
- Detailed project documentation
- High test coverage
- Likely leads to a reliable solution
- Avoids code duplication

Disadvantages

- Changing created test cases → heavy maintainance
- Slower initial pace
- Probable that test cases get written that are not needed
- May not work for exploratory projects

Testing Frameworks

Testing Frameworks

- Frameworks provide convenient tools to create tests
- Mechanism included to run the tests automatically and report results
- We will look into the built-in unittest library, pytest and doctest

unittest

- Built into the standard Python library

pytest

- Automatically discovers tests in your codebase
- Can run unittest
- Detailed information on failures
- Many external plugins available

doctest

- Built into the standard Python library
- Simple tests based on docstrings

Unit Tests with unittest

- Checks small piece of code, to ensure they work correctly (e.g., individual functions)
- The module unittest comes pre-installed with Python
- Do not ensure different functions / classes work together!

Trivial example: we want to test the following function

```
def add(a, b):  
    return a + b
```

Unit Tests

```
#test.py
import unittest

def add(a, b):
    return a + b

class TestAddFunction(unittest.TestCase):
    def test_add_positive_numbers(self):
        self.assertEqual(add(1, 2), 3)

    def test_add_negative_numbers(self):
        self.assertEqual(add(-1, -2), -3)

    def test_add_mixed_numbers(self):
        self.assertEqual(add(1, -2), -1)
        self.assertEqual(add(-1, 2), 1)

if __name__ == '__main__':
    unittest.main()
```

Output:

Ran 3 tests in 0.000s
OK

Unit Tests

```
#test.py
import unittest

def add(a, b):
    return a + b

class TestAddFunction(unittest.TestCase):
    def test_add_positive_numbers(self):
        self.assertEqual(add(1, 2), 1)

    def test_add_negative_numbers(self):
        self.assertEqual(add(-1, -2), -3)

    def test_add_mixed_numbers(self):
        self.assertEqual(add(1, -2), -1)
        self.assertEqual(add(-1, 2), 1)

if __name__ == '__main__':
    unittest.main()
```

Let's intentionally change the test to see what happens when a test fails:

```
..F
=====
FAIL: test_add_positive_numbers
(__main__.TestAddFunction.test_add_positive_numbers)
-----
Traceback (most recent call last):
  File "test.py", line 9, in test_add_positive_numbers
    self.assertEqual(add(1, 2), 1)
AssertionError: 3 != 1

-----
Ran 3 tests in 0.001s

FAILED (failures=1)
```

Assert methods in unittest

Some useful assert methods. Full list:

[unittest — Unit testing framework — Python 3.14.3 documentation](https://docs.python.org/3.14.3/library/unittest.html)

Method	Check
<code>assertEqual(a, b)</code>	<code>a == b</code>
<code>assertNotEqual(a, b)</code>	<code>a != b</code>
<code>assertTrue(x)</code>	<code>bool(x) is True</code>
<code>assertIs(a, b)</code>	<code>a is b</code>
<code>assertIsNone(x)</code>	<code>x is None</code>
<code>assertIsNotNone(x)</code>	<code>x is not None</code>
<code>assertIn(a, b)</code>	<code>a in b</code>
<code>assertNotIn(a, b)</code>	<code>a not in b</code>
<code>assertIsInstance(a, b)</code>	<code>isinstance(a, b)</code>
<code>assertIsSubclass(a, b)</code>	<code>issubclass(a, b)</code>

Method	What it checks	Input type
<code>assertIsInstance(a, b)</code>	Is object <code>a</code> an instance of class <code>b</code> ?	object
<code>assertIsSubclass(a, b)</code>	Is class <code>a</code> a subclass of class <code>b</code> ?	class

How to use unittest

It contains both a testing framework and a test runner

To set up a test, you:

- Put tests into classes as methods
- Use assertion methods as shown on the last slide in the `unittest.TestCase` class

Doctest

- Tests in doctests are written into the docstring of the function
- Formatted like an interactive Python session
- Describe commands and expected outputs

Example:

```
def square(x):  
    """ returns the square of x for any value.  
    >>> square(10)  
    100  
    >>> square(3.1415)  
    9.86902225  
    """  
    return x * x
```

Running it:

```
python -m doctest <filename>
```

Tests all defined doctests in the file. If successful, nothing is printed („No news is good news“).

If you want more information provided:

```
python -m doctest -v <filename>
```

Doctest

- Tests in doctests are written into the docstring of the function
- Formatted like an interactive Python session
- Describe commands and expected outputs

```
*****
File "C:\Users\ma1200408\VScodeprojects\AP_Examples\Unit_5\doctest_square.py", line 5, in doctest_square.square
Failed example:
    square(3.1415)
Expected:
    9.8690222
Got:
    9.86902225
*****
1 items had failures:
  1 of  2 in doctest_square.square
***Test Failed*** 1 failures.
```

```
python -m doctest -v <filename>
```

pytest Library

- Not included in the standard library. Install in your local environment with

```
python -m pip install pytest
```

- Offers powerful toolset
- Can run all tests in a project directory with

```
pytest
```

to discover tests, it looks for any files matching the pattern

- test_<name>.py
- <name>_test.py
- To test a single file, just specify the file. pytest will look for functions with names test_<name>()

Where to put our tests?

- Typically in a separate file (or files, depending on project complexity). We need to import the classes and functions we test
- Test function names should be descriptive of
 - Which function is being tested
 - What is being tested
- Test functions are of the form `test_<name>()` without any parameters

asserts – What's inside of the tests?

- Common statement to check the function with known input/output pairs:

```
assert <some boolean expression>
```

- E.g.,

```
def test_sqrt_valid():  
    assert sqrt(144) == 12
```

Multiple asserts:

- All must pass for a successful test.
- Test will stop at the first failure

```
def test_math():  
    assert 2 + 2 == 4      #  passes  
    assert 3 * 3 == 9      #  passes  
    assert 10 / 2 == 3     #  fails here  
    assert 5 - 1 == 4      #  never runs
```

Exceptions in pytest

- We can also check if exceptions are raised as expected
- raises() function by pytest

```
def square_root(x):  
    if x < 0:  
        raise ValueError("Negative numbers not allowed")  
    return sqrt(x)  
  
def test_square_root_raises_exception():  
    with pytest.raises(ValueError):  
        square_root(-144)
```

Be careful when comparing floating point results

- Floating points have limited precision. Sometimes seemingly harmless comparisons can go wrong, e.g., try:

```
0.1 + 0.2 == 0.3
```

- Can we check if the results are reasonably close?
- Yes! pytest offers an `approx()` function

```
0.1 + 0.2 == pytest.approx(0.3)
```


Testing with Dummy Data

- In many functions, you may need inputs to actually run a test
- Such inputs you create for tests is called a **fixture**
- **fixture** is a function that provides setup data or resources to your tests.
- Passing different values to the same test is called **parametrization**
- Both can be achieved with decorators („@“)

pytest - Parametrization

- Decorator to provide parameters and expected output

```
@pytest.mark.parametrize("a, b, expect", [
    ([1, 2, 3], [4, 5, 6], [5, 7, 9]),
    ([-1, -5, -3], [-4, -3, 0], [-5, -8, -3]),
])
def test_add_arrays(a, b, expect):
    output = add_arrays(a, b)

    assert output == expect
```

pytest - Fixtures

- As with parametrization, we want to split the data used in a test from the code of the test
- We might want to use the same data across multiple function tests

```
@pytest.fixture
def pair_of_lists():
    return [1, 2, 3], [4, 5, 6]

def test_add_arrays(pair_of_lists):
    a, b = pair_of_lists
    expect = [5, 7, 9]

    output = add_arrays(a, b)

    assert output == expect
```

Test Scenarios: Happy Path and Edge Cases



Happy Path

- The 'ideal' scenario with valid, expected input
- User does everything the system was designed for
- Critical to test first — must always work

Examples

`divide(10, 2) == 5`

`login('alice', 'correct_pw') → 200 OK`



Edge Cases

- Boundary values, unexpected or extreme inputs
- Where bugs hide, often missed in manual testing
- Think: What can possibly go wrong?

Examples

`divide(10, 0) → ZeroDivisionError?`

`login("", "") → rejected, not crash`

Test Coverage Illusions

- How much of your code is covered by tests? → Measures how many lines of code are executed during the tests.
- Branch coverage: Are all paths covered? Branch coverage can differ a lot from line coverage!
- Missing assert functions in tests lead to passing test without indication of correctness.
- Mock-data may not be representative of real cases, integration testing is crucial!
- Put a lot of effort in finding meaningful edge cases.
- Measure coverage, e.g., **for a package calculator**:

```
pytest --cov=calculator --cov-report=term-missing
```

```
pytest --cov=calculator --cov-report=term-missing --cov-report=html
```

- Hinweis: „covtest“ muss zuvor installiert werden:

```
pip install pytest-cov
```

Example:

- If your file has 100 lines
- Tests execute 80 lines → **80% coverage**

 What the HTML report shows

-  Green lines → code that was executed
-  Red lines → code that was NOT tested
-  Coverage percentages per file and overall

Test Case Template

1. Test case ID – unique identifier (e.g., TC_001)
2. Test case title/description – What is the test about?
3. Preconditions: Requirements before executing the test
4. Test steps: Actions to perform
5. Test data/input
6. Expected result
7. Actual result
8. Status – pass or fail
9. Comments – Additional notes or defect found

Example Test Case „Login“

Field	Details
Test case ID	TC_001
Test case title/description	Verify that a user can log in with valid username and password
Preconditions	- User is registered - Login page is accessible
Test steps	1. Open login page 2. Enter username john123 3. Enter password Password@123 4. Click Login
Test data/input	Username: john123 Password: Password@123
Expected result	User is logged in successfully and dashboard is displayed
Actual result	User is logged in successfully and dashboard is displayed
Status	Pass
Comments	No issues found

Project task

- Define test cases for your own projects
- Adhere to the provided template